

Parte A:

1. Cuando el arreglo ya está ordenado, Insertion Sort únicamente realiza una comparación por cada elemento para confirmar que no debe desplazarse. Esto resulta en exactamente 999 comparaciones y 0 intercambios. En cambio, Selection Sort realiza siempre realiza comparaciones sin importar el estado inicial de los datos, y Bubble Sort (aun optimizado con bandera) requiere realizar al menos la misma cantidad de comparaciones lineales, pero con mayor sobre costo operacional por iteración.
2. Un conjunto de datos que ya esté estrictamente ordenado (ascendente o descendente), como [10, 20, 30, 40, 50]. Al tomar el primer elemento, la partición izquierda siempre queda vacía y la derecha retiene n-1 elementos.
3. Conviene ordenar primero y usar búsqueda binaria. Buscar secuencialmente 100 veces sobre 10,000 registros. Si ordenamos primero con un algoritmo eficiente y luego aplicamos búsqueda binaria 100 veces. El costo inicial de ordenar se amortiza por completo gracias a la eficiencia de la búsqueda binaria.

Parte B:

Caso Evaluado	Valor a buscar	Búsqueda Secuencial (Comparaciones)	Búsqueda Binaria (Comparaciones)
Primer elemento	100001	1	4
Elemento del medio	200016	16	5
Ultimo elemento	400030	30	5
No existe	25	30	4

Parte C:

Algoritmo	Estado de los datos	Comparaciones	Intercambios
Bubble Sort	Ya ordenados	29	0
Bubble Sort	Invertidos	435	435
Selection Sort	Ya ordenados	435	0
Selection Sort	Invertidos	435	15
Insertion Sort	Ya ordenados	29	0
Insertion Sort	Invertidos	435	435

Explicación de las Diferencias

- Bubble Sort: Con datos ya ordenados, la bandera swapped detecta en la primera pasada que ningún elemento necesitó intercambio, reduciendo el trabajo a una sola iteración de comparaciones. Con datos desordenados, realiza el ciclo completo de validaciones e intercambios masivos.
- Selection Sort: Su comportamiento es totalmente insensible al estado inicial de la lista. Siempre recorre toda la estructura buscando el elemento menor de forma ciega, por lo que las

comparaciones se mantienen fijas en 435 tanto si está ordenado como si está desordenado. La única diferencia radica en que con datos ordenados no efectúa ningún intercambio.

- Insertion Sort: Con datos ya ordenados, cada elemento nuevo se compara únicamente con su predecesor inmediato y descubre que ya está en su lugar correcto, logrando una eficiencia lineal de $N - 1$ comparaciones y cero movimientos. Con datos invertidos, cada elemento debe desplazarse hasta el extremo opuesto, disparando al máximo tanto las comparaciones como los intercambios.

Parte D:

Insertion Sort presenta una complejidad cuadrática debido al uso de bucles anidados, lo que provoca que los tiempos crezcan drásticamente y se cuadrupliquen al duplicar el tamaño de la entrada. En contraste, Merge Sort implementa la técnica de "divide y vencerás" para partir recursivamente el arreglo en mitades que le permite procesar grandes volúmenes de datos de manera mucho más rápida y controlada.

Parte E:

Si tomamos la función de búsqueda secuencial (`busquedaSecuencialContador`) del código y medimos cuántas comparaciones hace en el peor de los casos (cuando el elemento no está en la lista) duplicando los datos, la tabla nos queda así

Tamaño (N)	Comparaciones (Peor Caso)	Factor ($T_{actual} \backslash T_{anterior}$)
1000	1000	-
2000	2000	2
4000	4000	2

Interpretación

Si te fijas en la tabla, el factor da exactamente 2 cada vez que duplicamos el tamaño de la entrada. Esto significa que, si duplicas la cantidad de datos, el número de comparaciones también se duplica de forma exacta y proporcional. Como el factor dio dos, podemos confirmar que es un algoritmo de orden $O(n)$, lo cual coincide perfectamente con lo que esperábamos: como la búsqueda secuencial revisa los elementos uno por uno, su esfuerzo crece al mismo ritmo que crece la lista.

Parte F:

No estoy de acuerdo. La afirmación es incorrecta porque reduce la Integración Continua y la Entrega Continua (CI/CD) exclusivamente a la fase de compilación de binarios. Aunque C++ requiere compilar, CI/CD abarca mucho más que eso: automatiza la ejecución de pruebas unitarias, verifica estilos de código (linting), detecta errores de regresión, analiza vulnerabilidades de seguridad y asegura que cualquier cambio realizado en el repositorio (git push) funcione de manera integrada antes de ser fusionado o desplegado. Python se beneficia enormemente de estas prácticas para garantizar la calidad del software.

Implementación:

Git Push → Ejecutar programa → Pruebas → Compilar → Desplegar